

Titolo della Tesi

Università degli Studi di Verona

Alberto Dosso

11 agosto 2026

Indice

1	Introduzione	1
2	Dettagli tecnici	2
2.1	Rete Neurale	2
2.1.1	Layers	3
2.1.2	Pesi	3
2.1.3	Bias	3
2.2	Dataset	4
2.2.1	Qualità di un Dataset	4
2.3	Training	5
2.3.1	Training from scratch	5
2.3.2	Transfer Learning	5
2.4	Epoca	6
2.5	Pytorch	7
2.5.1	Tensori	7
2.5.2	Grafici	7
3	YOLO	8
3.1	YOLO26	9
3.1.1	Versioni	9
3.1.2	Metriche di valutazione	9
3.2	Script	12
3.2.1	Dataset	12
3.2.2	Training	12
3.2.3	Risultati	13
3.2.4	Inferenza	16
4	SAM3	17
4.1	Generazione del Dataset	18
4.2	Script	18
4.2.1	Librerie	19

4.2.2	Configurazione	19
4.2.3	Main	20
4.2.4	Ottimizzazioni	21
4.3	Conclusioni	21
5	Segformer	23
6	VLM	25
6.1	Introduzione	25
6.2	Qwen 3	26
6.2.1	Training	26
6.3	Gemma 4	29
6.3.1	Risultati	29
7	Grounding DINO	30
	Bibliografia	31

1. Introduzione

L'obiettivo di questa ricerca è quello di analizzare e migliorare il rilevamento di capi d'abbigliamento all'interno di un'immagine utilizzando modelli di Machine Learning.

In questi ultimi anni l'avanzamento delle tecnologie riguardanti il campo dell'Intelligenza Artificiale ha avuto una crescita esponenziale, migliorando la comprensione che loro hanno del mondo, e la risoluzione di problemi complessi soprattutto grazie all'incremento della mole di dati.

La proposta è quella di usare modelli nel campo della visione artificiale, uniti ad algoritmi di Deep Learning, per migliorare il più possibile il riconoscimento di diverse le categorie di capi d'abbigliamento.

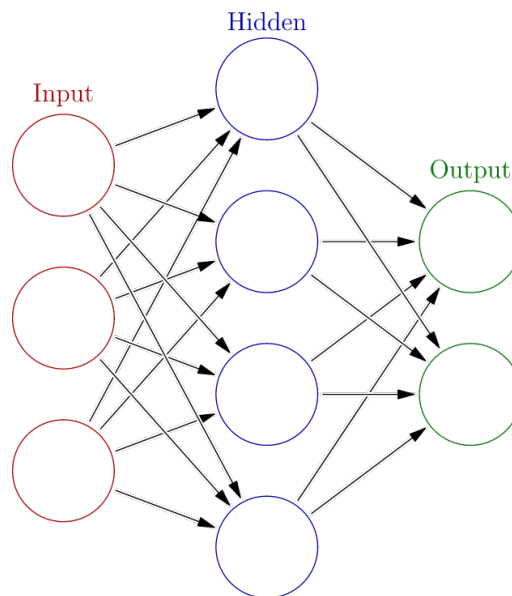
2. Dettagli tecnici

2.1 Rete Neurale

Una Rete Neurale è un modello matematico che simula il funzionamento della mente biologica umana, ed è utilizzata per tentare di risolvere problemi ingegneristici di intelligenza artificiale. [1]

Nel nostro caso, per la risoluzione di problemi di visione artificiale, utilizzeremo le reti neurali multi livello (o multi layer), chiamate anche MLP (Multi Layer Perceptron), che appartengono ad una sotto categoria del Machine Learning chiamata Deep Learning.

Da notare che non tutti i modelli di Intelligenza Artificiale sono basati sulle reti neurali; esistono anche altre tecnologie utilizzate nel mondo del Machine Learning come i Decision Trees, i quali creano diagrammi di flusso basati su condizioni specifiche, oppure le Random Forest, cioè l'unione di più Decision Tree indipendenti.



[2]

Figura 2.1: Percettrone Multistrato

2.1.1 Layers

In una Rete Neurale i layers rappresentano i diversi strati della rete. In ogni strato sono presenti uno o più neuroni chiamati unità.

Sono generalmente presenti tre strati :

1. Input : con unità che rappresentano i campi di input.
2. Hidden Layers : con uno o più strati.
3. Output : con una o più unità che rappresentano i campi obiettivo.

I dati di input vengono presentati nel primo strato e i valori vengono propagati da ciascun neurone a ogni neurone nello strato successivo, ed alla fine viene fornito un risultato nello strato di output. [3]

2.1.2 Pesì

Le unità sono connesse con varie intensità di connessione, dette pesi.

Tutti i pesi inizialmente sono casuali e le risposte ottenute dalla rete risulteranno probabilmente prive di senso.

La rete apprende generando una previsione per ogni input dato e correggendo i pesi ogni volta che viene sbagliata una previsione. Questo processo, anche detto *epoca*, viene ripetuto molte volte e la rete continua a migliorare le sue previsioni finché uno o più criteri di interruzione non vengono soddisfatti. [3]

2.1.3 Bias

Il parametro di bias è quello che, durante il calcolo matematico che avviene all'interno della singola unità (neurone) permette di aggiungere un'errore al modello, considerato troppo semplice, con l'obiettivo di catturare una maggiore complessità dei dati.

2.2 Dataset

Un Dataset è una raccolta strutturata di dati, che in questo caso servirà per la fase di addestramento (training) dei nostri modelli.

In generale, un Dataset per l'addestramento di Reti Neurali è suddiviso in :

- Training set : immagini usate per il training del nostro modello.
- Validation set : immagini utilizzate durante l'addestramento, per valutare le prestazioni del modello su dati non visti e per effettuare la messa a punto dei parametri.
- Test set : un insieme di immagini utilizzate alla fine dell'addestramento, per valutare le prestazioni finali del modello, su dati completamente nuovi che non sono stati utilizzati per il training.

Il Validation set è opzionale, ma molto utile per migliorare la qualità del training del modello.

2.2.1 Qualità di un Dataset

La qualità di un Dataset è data dalla correttezza delle sue annotazioni, cioè dalla coerenza di quello che mi stanno dicendo i dati al suo interno, ma anche dalla varietà dei dati al suo interno, perché avrò molti più esempi da mostrare al mio modello nella fase di addestramento.

Perciò se i dati sono coerenti ai problemi del mondo reale di cui si occuperà il modello, impararne i pattern e le correlazioni consentirà al modello addestrato di fare previsioni accurate su nuovi dati.

2.3 Training

L'addestramento di un modello è la fase del Machine Learning (ML) in cui avviene l'apprendimento. Nel ML, l'apprendimento implica la regolazione dei parametri di un modello ML. Questi parametri includono *pesi* e *bias* nelle funzioni matematiche che costituiscono i loro algoritmi.

Durante questa fase, ogni volta che un input arriva ad un nodo, viene moltiplicato per il peso (weight) associato a quella specifica connessione, e la somma di tutti questi input con un certo valore di bias ne determinerà poi i valori in output.

Dal punto di vista matematico, l'obiettivo di questo apprendimento è quello di ridurre al minimo una funzione di perdita che quantifica l'errore degli output nelle attività di addestramento. Quando l'output della funzione di perdita scende al di sotto di una soglia predeterminata, ovvero l'errore del modello nelle attività di addestramento è sufficientemente piccolo, il modello viene considerato "addestrato". [4]

2.3.1 Training from scratch

In questa tipologia di training, il modello viene addestrato su dati annotati senza partire da pesi pre-trainati, quindi senza usare come base un modello già allenato.

Partendo da zero, ovvero from scratch, i pesi non sono definiti ma vengono assegnati dei numeri casuali che, soltanto durante il processo di training, verranno modificati per ottimizzare le prestazioni.

Questo tipo di training è valido quando abbiamo a disposizione un numero considerevole di dati annotati e l'utilizzo del modello è molto specifico.

2.3.2 Transfer Learning

Nel caso in cui ci trovassimo a lavorare con un dataset limitato, o nel caso in cui l'obiettivo sia quello di riuscire a risparmiare tempo e risorse, sarebbe possibile partire da un modello già allenato, cioè da un modello su cui è già stata fatto il training.

L'idea è quella di "trasferire" ad un modello provvisto di pesi e con dei parametri di precisione già significativi, la parte di conoscenza necessaria per riconoscere i dati annotati nel nostro dataset.

Si inizia quindi dal modello pre-trainato e lo si adatta (fine-tuning) al nostro target, spesso congelando i primi layers ed ottimizzando solamente quelli finali.

2.4 Epoca

Un'epoca rappresenta un passaggio completo del dataset attraverso l'algoritmo in fase di learning, è un concetto fondamentale nel training delle reti neurali, che permette al modello di imparare continuando costantemente a rivedere le stesse immagini, o i medesimi elementi del dataset.

Il numero di epoch si specifica solitamente in ogni procedura di training e determina quante volte il modello imparerà dal set di train del dataset, influenzando le performance e la qualità del modello.

Scegliere il numero di epoch corretto è molto importante perché se viene scelto in maniera errata può creare due problemi:

- Underfitting: il modello non è stato trainato per un numero sufficiente di epochs. Le performances non sono adeguate né sul set di training né su quello di test.
- Overfitting: il modello è stato trainato per troppi epochs. In questo caso memorizza i dati di training in maniera troppo approfondita e perde le sue abilità di analizzare i nuovi dati. Ha una precisione eccellente sul training set ma scarsa sul dataset di validazione.

Una tecnica comune per evitare l'overfitting è quella dell'early stopping, nella quale il training viene fermato quando le performances sul set di validazione smettono di migliorare. A volte è possibile specificare un parametro chiamato patience, che specifica dopo quanti epochs "non migliorativi" il training deve fermarsi. [5].

2.5 Pytorch

PyTorch è un framework di deep learning open source basato su software usato per creare reti neurali. Combina la libreria di machine learning (ML) di Torch con un'API di alto livello basata su Python. La sua flessibilità e facilità d'uso, tra gli altri benefici, lo hanno reso il principale framework di ML per le comunità accademiche e di ricerca.

2.5.1 Tensori

In qualsiasi algoritmo di machine learning, anche quelli applicati a informazioni apparentemente non numeriche come suoni o immagini, i dati devono essere rappresentati numericamente. In PyTorch ciò viene ottenuto tramite tensori che fungono da unità fondamentali di dati utilizzate per il calcolo sulla piattaforma.

Dal punto di vista linguistico, “tensore” è un termine generico che si riferisce ad alcune entità matematiche più familiari come :

- Un vettore è un tensore monodimensionale contenente più scalari dello stesso tipo. Una tupla è un tensore monodimensionale contenente diversi tipi di dati.
- Una matrice è un tensore bidimensionale contenente più vettori dello stesso tipo.
- I tensori con tre o più dimensioni, come i tensori tridimensionali usati per rappresentare le immagini RGB negli algoritmi di computer vision, sono collettivamente denominati tensori N-dimensionali.

Oltre a codificare gli input e gli output di un modello, i tensori di PyTorch codificano i parametri del modello: pesi e gradienti che vengono ”appresi” nella machine learning.

2.5.2 Grafici

I grafici di calcolo dinamico (DCG) sono il modo in cui i modelli di deep learning vengono rappresentati in PyTorch. Nel contesto del deep learning, questi grafici traducono essenzialmente il codice di una rete neurale in un diagramma di flusso che indica le operazioni eseguite in ciascun nodo e le dipendenze tra diversi strati nella rete, cioè la disposizione di fasi e sequenze che trasformano i dati di input in dati di output. [6]

3. YOLO

YOLO, ovvero "You Only Look Once", è un diffuso algoritmo di rilevamento di oggetti in tempo reale.

YOLO unifica quello che un tempo era un processo a più fasi, utilizzando un'unica rete neurale per eseguire sia la classificazione sia la previsione dei riquadri di delimitazione (bounding box) degli oggetti rilevati. Ogni bounding box è definita dalle coordinate che ne identificano posizione e dimensioni all'interno dell'immagine, oltre a un punteggio di confidenza e all'etichetta della classe dell'oggetto. In questo modo il modello è in grado di localizzare e classificare simultaneamente tutti gli oggetti presenti in una singola inferenza, formulando il problema come una regressione diretta delle coordinate dei bounding box e delle probabilità di appartenenza alle diverse classi.

È fortemente ottimizzato per le prestazioni di rilevamento e può operare molto più velocemente rispetto all'esecuzione di due reti neurali distinte per rilevare e classificare gli oggetti separatamente. Ciò avviene riadattando i classificatori di immagini tradizionali per la specifica attività di regressione volta a individuare i bounding box degli oggetti. [7]

3.1 YOLO26

YOLO26 è l'ultima famiglia unificata di modelli di visione in tempo reale. Introduce l'inferenza nativa end-to-end, una head di rilevamento più leggera, una ricetta di addestramento aggiornata e head specifiche per compito per rilevamento, segmentazione, stima della posa, classificazione. [8]

3.1.1 Versioni

I modelli YOLO26, offrono diversi tagli per ogni versione del modello, passando dalla più piccola denominata "nano" (YOLO26n), alla più grande "extra large" (YOLO26x), nei quali differenza sta nella loro dimensione a livello di parametri. Questa variazione di parametri ci permetti di aumentare le prestazioni, e quindi la qualità del modello.

3.1.2 Metriche di valutazione

Dopo aver addestrato un modello YOLO, dobbiamo misurarne le prestazioni ed eventualmente perfezionarlo per colmare le lacune. La valutazione utilizza metriche come precision e la recall, le quali unite alla mAP ci permettono di quantificare l'accuratezza del modello.

Model	mAP ₅₀	mAP ₉₅	Speed	Speed	params (M)
			CPU (ms)	GPU (ms)	
YOLO26n	40.9	40.1	38.9 ± 0.7	1.7 ± 0.0	2.4
YOLO26s	48.6	47.8	87.2 ± 0.9	2.5 ± 0.0	9.5
YOLO26m	53.1	52.5	220.0 ± 1.4	4.7 ± 0.1	20.4
YOLO26l	55.0	54.4	286.2 ± 2.0	6.2 ± 0.2	24.8
YOLO26x	57.5	56.9	525.8 ± 4.0	11.8 ± 0.2	55.7

[8]

Precision

La precision è la frazione di istanze rilevanti tra le istanze recuperate.

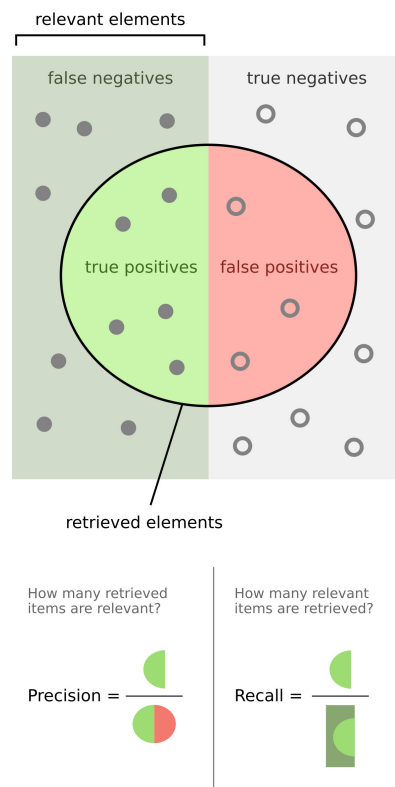
$$Precision = \frac{\text{Istanze rilevanti recuperate}}{\text{Tutte le istanze recuperate}} = \frac{TP}{TP + FP}$$

Recall

La recall (anche nota come sensibilità) è la frazione di istanze rilevanti che sono state recuperate.

$$Recall = \frac{\text{Istanze recuperate rilevanti}}{\text{Tutte le istanze rilevanti}} = \frac{TP}{TP + FN}$$

Sia la precision che la recall si basano sull'output generato dal nostro modello. Queste metriche possono essere meglio rappresentate tramite questa immagine.



[9]

F1 score

L'F1 score mette in rapporto tra la precision e la recall, offrendoci una migliore comprensione del bilanciamento del nostro modello.

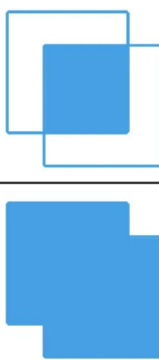
mAP

La precisione media (mean average precision) è una metrica di apprendimento automatico progettata per valutare gli algoritmi di rilevamento degli oggetti.

Negli articoli scientifici relativi al rilevamento di oggetti, è possibile imbattersi in abbreviazioni come mAP_{50} o mAP_{95} . In breve, questa notazione indica la soglia IoU utilizzata per calcolare l'mAP.

IoU

L'Intersection over Union viene calcolato dividendo la sovrapposizione tra l'annotazione prevista e quella reale per l'unione di queste due.

$$\text{IoU} = \frac{\text{Area of Overlap}}{\text{Area of Union}}$$


[10]

Figura 3.1: Intersection over Union

Per capire meglio, ecco alcuni esempi :

- Se impostassimo la soglia IoU a 0,9, e la precision fosse pari al 16% vorrà dire che solo 1 previsione su 6 corrisponde al punteggio;
- Con una soglia pari a 0,7, ed una precision del 65% avrò che 4 previsioni sono al di sopra di tale punteggio.
- E se la soglia fosse del 0,3, e la precision salisse al 100% vuol dire che tutte le previsioni hanno un IoU superiore a 0,3.

Pertanto, la soglia IoU può influenzare significativamente il valore medio finale della Precisione Media. Questa dipendenza introduce variabilità nella valutazione del modello. Questo è negativo, perché può verificarsi uno scenario in cui un modello ottiene buoni risultati con una determinata soglia IoU e prestazioni nettamente inferiori rispetto ad un'altra.

Per contrastare questa debolezza, si è deciso di misurare la Precisione Media per ogni classe e per ogni soglia IoU, per poi calcolare la media dei punteggi ottenuti.

[11]

3.2 Script

Di seguito sono definite le operazioni necessarie, e specifiche per l'addestramento del modello base per la object detection YOLO26n.

3.2.1 Dataset

Il dataset utilizzato per il training di questo specifico modello YOLO è composto da una serie di immagini contenenti vari capi d'abbigliamento. Nello specifico, le annotazioni di questo dataset sono state generate tramite un ulteriore modello di segmentazione, chiamato SAM3, che verrà approfondito in seguito.

Per la fase di inferenza è stato invece impiegato un dataset più ridotto. Poiché quest'ultimo comprendeva inizialmente alcune immagini già presenti nel set di training, è stata effettuata un'operazione di pulizia dei dati: tali immagini sono state rimosse dal dataset di addestramento, garantendo così una maggiore affidabilità nella valutazione qualitativa del modello.

3.2.2 Training

Inizializzazione

```
1 import ultralytics
2 from ultralytics import YOLO
```

Elaborazione

```
1 model = YOLO("yolo26n.pt")
2
3 trainArgs = {
4     'data' : "dataset/dataset.yaml",
5     'epochs' : 100,
6     'imgsz' : 640,
7     'batch' : 64,
8     'device' : 0,
9     'name' : "detection",
10    'workers' : 4,
11    'patience' : 10,
12    'plots' : True,
13 }
14
15 model.train(**trainArgs)
```

Parametri

Dei parametri utilizzati nell'addestramento, i più rilevanti per le prestazioni sono :

- epochs : numero di epoche
- batch size : numero di immagini alla volta che gli passo
- device : hardware che voglio utilizzare
- workers : numero di "lavoratori" CPU che mi aiutano in questo processo

3.2.3 Risultati

Alla fine della fase di addestramento, verranno prodotte delle statistiche che ci serviranno per la valutazione del nostro modello, cioè la precision, recall, F1 score e l'mAP.

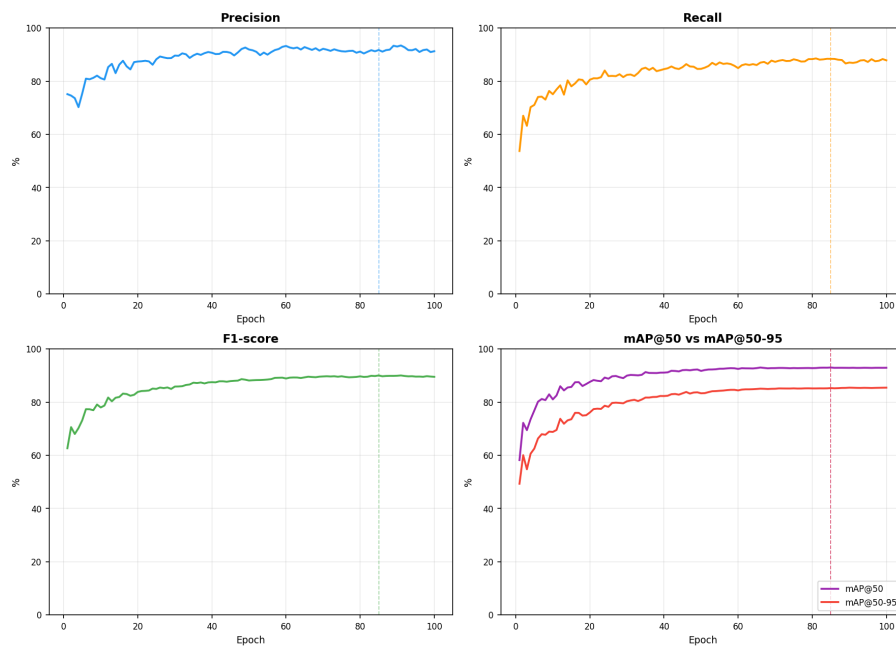
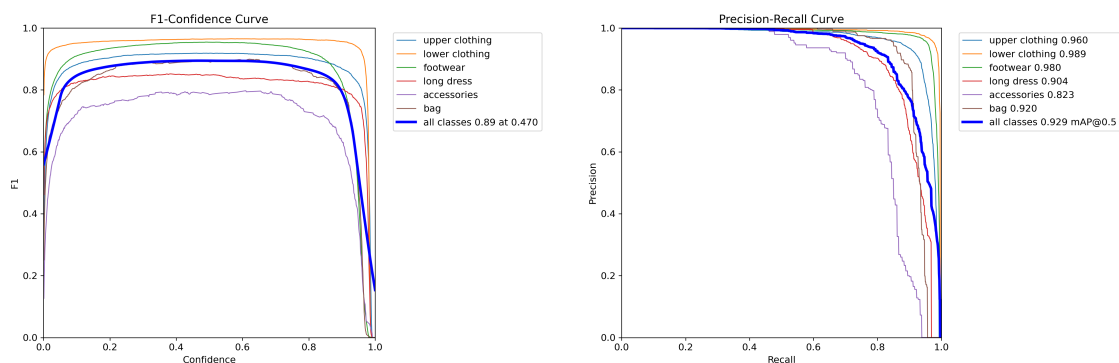


Figura 3.2: statistiche

Possiamo notare come inizialmente i grafici siano più rumorosi, e come invece più si va avanti nel numero di epoche, più la curva si stabilizzerà raggiungendo il suo plateau.

Confidence

È il livello minimo di confidenza entro il quale il modello considera l'oggetto come valido. Questo è ottenuto andando a guardare il risultato dell'IoU e controllando che sia al sopra di questa soglia minima, che solitamente è impostata di default al 50%.



Nel grafico a sinistra possiamo vedere che il punto massimo della curva F1, la quale rappresenta il punto ottimale in cui la soglia di confidence fornisce il miglior compromesso tra precision e recall, è di 0.89 ottenuto con confidence 0.470.

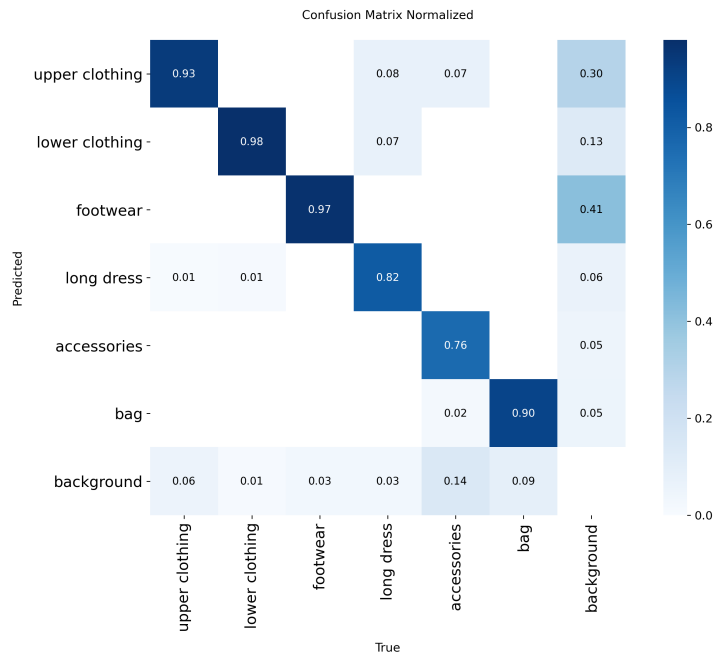
Quindi il miglior bilanciamento tra la precision e la recall si otterrà impostando una confidence, cioè un limite di soglia, pari al 47%. Questo grafico è molto importante per scegliere il valore di confidence da fornire in fase di inferenza, dove appunto si minimizzano il più possibile i fenomeni di falso positivo (precision) e falso negativo (recall).

In quello a destra, invece mostra la relazione fra precision e recall, con un valore di mAP50 pari a 0.929, e indica la media della precision agli intervalli di recall per tutte le classi, alla soglia di IoU di 0.5.

In sostanza evidenzia il bilanciamento tra la capacità del modello di identificare in maniera corretta oggetti (precision) e la capacità di trovarli tutti (recall) per i vari livelli di soglia di confidenza.

Confusion matrix

La confusion matrix mostra come il modello classifica le classi su cui è stato addestrato.



Il grafico può essere riassunto con alcune delle sue classi come :

- Predicted "upper clothing" | True "upper clothing" :
- il modello ha identificato correttamente il 93% della categoria "upper clothing".
- Predicted "background" | True "accessories" :
- il 14% degli accessori è stato classificato come "background" (falso negativo).
- Predicted "footwear" | True "background" :
- il 41% del "background" è stato classificato come tale (vero negativo).
- Predicted "background" | True "background" :
- nessun "background" è stato erroneamente classificato come capo d'abbigliamento (falso positivo assente).

3.2.4 Inferenza

Dopo aver addestrato il modello con il nostro dataset, è possibile procedere con l'inferenza su altre immagini per verificare la presenza di alberi. Importante è soprattutto fare l'inferenza su un dataset diverso da quello utilizzato nella fase di training, così da ottenere dei risultati più veritieri.

È importante assicurarsi che le immagini in input siano dello stesso formato di quelle preparate per il training, in questo caso 640x640, devono quindi rispettare la risoluzione e la composizione dei channels RGB.

Qui di seguito è dettagliata la funzione Python per l'inferenza con YOLO di Ultralytics, tramite la libreria OpenCV utilizzata in questo caso per lavorare con le immagini in input e output.

```
1 def run(modelPath, imagesDir, outputDir="predictions", conf=0.25):
2     model = YOLO(modelPath)
3     Path(outputDir).mkdir(parents=True, exist_ok=True)
4
5     for imgPath in Path(imagesDir).iterdir():
6         img = cv2.imread(str(imgPath))
7         if img is None:
8             continue
9
10        for result in model(img, conf=conf):
11            for box in result.boxes.xyxy.cpu().numpy().astype(int):
12                x1, y1, x2, y2 = box
13                cv2.rectangle(img, (x1, y1), (x2, y2), (0, 255, 0), 2)
14
15    cv2.imwrite(f"{outputDir}/{imgPath.stem}_result.jpg", img)
```

Questa funzione implementa la fase di inferenza del modello YOLO26 su un insieme di immagini. Dato il percorso dei pesi addestrati e quello della cartella di input, il modello viene caricato tramite la libreria Ultralytics e applicato iterativamente a ciascuna immagine, con una soglia di confidenza configurabile che filtra le predizioni meno affidabili.

Per ogni immagine vengono estratte le bounding box predette, sovrapposte all'immagine originale e il risultato viene salvato in una cartella di output dedicata, consentendo così una verifica qualitativa delle detection prodotte dal modello.

4. SAM3

Il Segment Anything Model 3 (SAM 3) è un modello foundation di computer vision sviluppato da Meta e integrato nella libreria Ultralytics, progettato per compiti di rilevamento, segmentazione di istanze (instance segmentation) e tracciamento di oggetti all'interno di sequenze video. Rispetto alle versioni precedenti (SAM 1 e SAM 2), le quali limitavano la segmentazione a un singolo oggetto individuato tramite prompt visivi puntuali (come coordinate o riquadri delimitatori), SAM 3 introduce il paradigma della Promptable Concept Segmentation (PCS). Grazie a questo approccio a vocabolario aperto (open-vocabulary), il modello è in grado di identificare e segmentare contemporaneamente tutte le istanze appartenenti a un determinato concetto visivo presente nella scena. Tali concetti possono essere specificati tramite descrizioni in linguaggio naturale (es. "gatto a strisce"), esemplari visivi di riferimento o combinazioni di entrambi.

Dal punto di vista architetturale, SAM 3 disaccoppia la fase di riconoscimento da quella di localizzazione mediante l'uso di un token di presenza globale. Il sistema separa nettamente il modulo di rilevamento — basato su un'architettura di tipo DETR con fusion encoder — dal tracker video con memoria ereditato da SAM 2. Addestrato sul vasto dataset SA-Co (comprendente oltre 5 milioni di immagini e 1,4 miliardi di maschere), SAM 3 raggiunge prestazioni allo stato dell'arte con un raddoppio dell'efficienza nella segmentazione per concetto e un tempo medio d'inferenza di circa 30 ms per immagine. [12]

4.1 Generazione del Dataset

Inizialmente si era partiti da un insieme primario di immagini privo delle annotazioni necessarie per l'addestramento dei modelli di object detection. Date le elevate capacità di generalizzazione e accuratezza di SAM 3, il modello è stato impiegato come strumento di annotazione automatica (pseudo-labeling) per la costruzione del dataset finale.

Nello specifico, la funzionalità PCS ha consentito di estrarre le regioni d'interesse fornendo in input al modello le immagini accompagnate da prompt testuali descrittivi delle varie categorie di capi d'abbigliamento da identificare. L'elaborazione ha prodotto per ciascuna immagine le relative maschere di segmentazione per le istanze rilevate.

Poiché l'obiettivo finale risiedeva nell'addestramento del modello di object detection YOLO26 — il quale richiede in input delimitatori rettangolari (bounding box) anziché maschere poligonali — si è proceduto alla conversione sintetica delle annotazioni: a partire dai contorni di ciascuna maschera generata da SAM 3, sono state calcolate le coordinate estreme per ricavare i bounding box corrispondenti. Tale procedura ha permesso di trasformare efficacemente un processo di segmentazione di istanze in un dataset strutturato per l'addestramento di modelli di object detection.

4.2 Script

Lo script realizza un annotatore automatico: prende in ingresso una cartella di immagini prive di etichette e produce in uscita un dataset di object detection in formato YOLO, senza alcun intervento manuale. Il ruolo di SAM3 è quello di annotatore zero-shot: non viene addestrato né adattato al dominio, ma viene interrogato in linguaggio naturale con i nomi delle classi che si vogliono ottenere. L'annotazione umana, che è la voce di costo dominante nella costruzione di un dataset, viene sostituita dalla definizione di una lista di prompt testuali.

Un punto concettuale importante è che SAM3 è un modello di segmentazione, quindi restituisce maschere a livello di pixel. La detection non è quindi un output nativo del modello, ma una proiezione: la maschera è la rappresentazione intermedia da cui si ricava la bounding box. Questo spiega perché lo stesso identico flusso possa alimentare, senza costi aggiuntivi di inferenza, anche un dataset di segmentazione semantica.

4.2.1 Librerie

```

1 import shutil
2 import torch
3
4 from pathlib import Path
5 from PIL import Image
6 from transformers import Sam3Model, Sam3Processor

```

4.2.2 Configurazione

Tutto ciò che definisce il dataset è dichiarativo: le cartelle di input e output, l'elenco delle classi e le soglie. L'ordine di CLASSES definisce implicitamente il class_id numerico usato nelle annotazioni YOLO.

CLASS_PROMPTS introduce una distinzione utile da sottolineare: la classe del dataset non coincide necessariamente con il prompt dato al modello. Categorie generali come "accessories" sono concetti astratti che il modello riconosce male; interrogandolo invece con termini concreti (hat, cap) la resa migliora sensibilmente, e tutte le maschere trovate confluiscono comunque nell'unica classe di destinazione.

```

1 IMAGES_DIR = Path("dataset/images")    # immagini senza etichette
2 OUTPUT_DIR = Path("output/detection")  # dataset generato
3
4 CLASSES = [
5     "upper clothing",
6     "lower clothing",
7     "footwear",
8     "long dress",
9     "accessories",
10    "bag",
11 ]
12
13 CLASS_PROMPTS = {
14     "accessories": ["hat", "cap", "scarf"],
15     "bag": ["handbag", "backpack"],
16 }
17
18 SCORE_THRESHOLD = 0.5    # confidenza minima delle maschere restituite da SAM3
19 IOU_MERGE_THRESHOLD = 0.5 # IoU >= soglia --> stesso oggetto
20 MIN_AREA_RATIO = 0.001  # scarta le maschere piccole
21
22 DEVICE = "cuda"
23 DTYPE = torch.float16    # tipo di dato

```

4.2.3 Main

Prepara la struttura delle cartelle e carica il modello una sola volta fuori dal ciclo e itera sulle immagini. Ogni immagine annotata produce una copia nella cartella images/ e il corrispondente file di etichette in labels/; le immagini in cui non viene rilevato nulla vengono semplicemente saltate e non entrano nel dataset.

```
1 def main():
2     images_out = OUTPUT_DIR / "images"
3     labels_out = OUTPUT_DIR / "labels"
4
5     for folder in (images_out, labels_out):
6         folder.mkdir(parents=True, exist_ok=True)
7
8     (OUTPUT_DIR / "classes.txt").write_text("\n".join(CLASSES) + "\n")
9
10    # caricamento del modello
11    model = Sam3Model.from_pretrained("facebook/sam3",
12                                       torch_dtype=DTYPE).to(DEVICE).eval()
13    processor = Sam3Processor.from_pretrained("facebook/sam3")
14
15    for path in sorted(IMAGES_DIR.iterdir()):
16        label_file = labels_out / f"{path.stem}.txt"
17
18        if RESUME and label_file.exists():
19            continue
20
21        image = Image.open(path).convert("RGB")
22        masks, labels = annotate(model, processor, image)
23        if not masks:
24            continue
25
26        width, height = image.size
27        shutil.copy2(path, images_out / path.name)
28        label_file.write_text("\n".join(to_yolo(masks, labels, width, height)) +
29                                   "\n")
30
31    print(f"{path.name}: {len(masks)} oggetti annotati")
```

4.2.4 Ottimizzazioni

Durante l'utilizzo di questo modello, si è notato un carico di computazione abbastanza importante, ottenendo delle tempistiche alquanto pessime, in media secondi ad immagine processata. Perciò questo tipo di problematica è stata risolta applicando principalmente due ottimizzazioni.

1. Riutilizzo dell'encoder visivo.

SAM3, come tutti i modelli della famiglia SAM, è composto da un encoder visivo pesante e da una testa di decodifica leggera condizionata dal prompt. L'encoder dipende solo dall'immagine, non dal testo: rieseguirlo per ogni prompt significherebbe ripetere inutilmente la parte più costosa del modello. Qui l'embedding visivo viene calcolato una sola volta per immagine, e poi passato a tutte le chiamate successive.

Con la configurazione attuale - sei classi per un totale di nove prompt - significa un solo passaggio dell'encoder invece di nove: il costo per immagine diventa una codifica visiva più nove decodifiche leggere, e cresce in modo quasi piatto all'aumentare del numero di classi. Questa è di fatto l'ottimizzazione che rende praticabile l'annotazione di dataset con molte categorie.

2. Inferenza in fp16.

Il modello è caricato in mezza precisione. Su GPU dotate di Tensor Core il guadagno è netto e, trattandosi di sola inferenza senza accumulo di gradienti, l'output è di fatto equivalente a quello in fp32.

4.3 Conclusioni

L'utilizzo di questo modello si è rivelato di grande aiuto nello sviluppo del nostro lavoro, sia per la creazione di nuovi dataset a partire dalle sole immagini, sia per l'elevata precisione garantita. Grazie alle sue capacità di segmentazione, è stato possibile automatizzare parte del processo di annotazione, riducendo i tempi di preparazione dei dati e migliorandone la qualità. La segmentazione consente di ottenere una rappresentazione più accurata rispetto a un semplice box, il quale include porzioni di sfondo e altri elementi non pertinenti, introducendo rumore nei dati. Questo approccio permette invece di identificare esclusivamente i pixel dell'oggetto di interesse, generando dataset più precisi e coerenti.

Come verrà mostrato nei capitoli successivi, il modello oggetto di questo studio otterrà risultati superiori rispetto agli altri modelli analizzati. Sebbene il confronto non sia perfettamente allineato sotto tutti gli aspetti metodologici, le pre-

stazioni ottenute confermeranno comunque l'efficacia dell'approccio basato sulla segmentazione.

5. Segformer

Negli ultimi anni, questo settore ha vissuto una profonda rivoluzione guidata dall'introduzione dei modelli Transformer. Originariamente concepiti per l'elaborazione del linguaggio naturale, questi modelli sono stati adattati con successo all'analisi delle immagini grazie al pionieristico Vision Transformer (ViT), presentato nel 2020. Il ViT ha dimostrato che è possibile analizzare le immagini scomponendole in una griglia di piccoli tasselli (patch) e trattandoli in modo simile alle parole di una frase. [13]

Sebbene il ViT abbia raggiunto risultati eccezionali, presentava limiti critici per la segmentazione: restituiva mappe a bassa risoluzione spaziale e richiedeva risorse di calcolo troppo elevate per immagini di grandi dimensioni. Per superare questi ostacoli e rendere la segmentazione altamente efficiente, la ricerca ha prodotto architetture di nuova generazione come SegFormer. [14]

SegFormer è un framework progettato per bilanciare efficienza e precisione senza appesantire la potenza di calcolo. Il cuore di questa architettura è il Mix Transformer (MiT), un encoder strutturato in modo gerarchico. A differenza del ViT originale, il MiT estrae informazioni a scale diverse, riducendo progressivamente la risoluzione dell'immagine mentre ne arricchisce il significato semantico. Inoltre, il MiT elimina la necessità di assegnare coordinate di posizione fisse ai pixel, consentendo al modello di adattarsi in modo naturale a immagini di qualsiasi dimensione.

Le informazioni estratte dall'encoder vengono poi inviate a un decoder con una struttura estremamente leggera basata esclusivamente su MLP. Evitando moduli di decodifica complessi, questo design snello fonde i dettagli geometrici dei primissimi livelli con la forte comprensione contestuale degli ultimi livelli.

Nonostante SegFormer sia molto efficiente, esso opera all'interno di un cosiddetto "vocabolario chiuso": la rete è programmata per riconoscere in modo automatico solo uno specifico e limitato insieme di categorie prestabilite durante l'addestramento. L'attuale frontiera della visione artificiale si sta invece muovendo verso i modelli open-vocabulary multimodali, il cui massimo esponente è SAM 3.

SAM 3 introduce una capacità innovativa nota come "Segmentazione di Concetti basata su Prompt". Mentre SegFormer richiede etichette numeriche predefinite, SAM 3 permette all'utente di rilevare, segmentare e tracciare qualsiasi oggetto o concetto guidando il modello tramite una frase di testo, un'immagine di esempio o un click del mouse. Questa versatilità estrema non si limita alle singole foto, ma si estende nativamente al tracciamento coerente lungo i fotogrammi di un video. [15]

In conclusione, mentre architetture snelle come SegFormer rimangono molto valide per applicazioni specifiche in tempo reale e con risorse computazionali limitate, i cosiddetti Foundation Models come SAM 3 estendono le potenzialità della segmentazione verso un vocabolario illimitato.

6. VLM

6.1 Introduzione

I modelli visivo-linguistici (VLM) rappresentano una grande evoluzione nella comprensione delle immagini. In generale, il loro funzionamento si basa sulla collaborazione in sequenza di tre elementi principali: il processo inizia con un modulo visivo che "osserva" l'immagine e ne cattura i dettagli essenziali; successivamente, un traduttore converte queste informazioni visive in un formato leggibile dall'intelligenza artificiale, creando un ponte tra le immagini e le parole. Infine, entra in azione il cervello linguistico, che riceve i dati visivi tradotti assieme alle richieste testuali dell'utente, combinandoli per elaborare una risposta discorsiva e di senso compiuto.

Questo flusso di lavoro integrato rappresenta un grande passo in avanti rispetto alla computer vision classica, ma è importante fare una distinzione rispetto ai modelli tradizionali ultra-specializzati, come YOLO o SAM. Essendo focalizzati su un unico obiettivo visivo, questi algoritmi tradizionali rimangono nettamente superiori in termini di precisione geometrica: riescono a localizzare e isolare gli elementi nella foto con un'accuratezza che i VLM, dovendo gestire simultaneamente la visione e l'elaborazione del linguaggio, non possono eguagliare.

La vera forza dei VLM, tuttavia, non risiede nel ritaglio geometrico perfetto, ma nella profonda comprensione semantica della scena. Mentre modelli come YOLO o SAM si limitano a individuare meccanicamente la presenza isolata all'interno della foto, un VLM utilizza il ragionamento logico per interpretare il contesto, deducendo e spiegando a parole quello che avviene all'interno dell'immagine. In questo modo, i VLM superano la rigidità dei compiti tradizionali, unendo la semplice percezione visiva alla capacità di ragionare e dialogare.

6.2 Qwen 3

Creato da Alibaba Cloud, Qwen3-VL è un modello costruito per elaborare contemporaneamente un'enorme quantità di informazioni. La sua forza sta nel riuscire a gestire testi lunghissimi mescolati a decine di immagini e video, senza mai perdere il filo del discorso. [16]

Per fare questo, utilizza due soluzioni intelligenti: la prima gli permette di avere un senso dell'orientamento eccezionale, capendo perfettamente dove si trova un oggetto nello spazio di una foto o l'esatto ordine cronologico in un video; la seconda lo aiuta a non perdersi nemmeno i dettagli visivi più piccoli, unendoli al testo in modo coerente. Grazie a queste abilità avanzate, Qwen3-VL può svolgere compiti pratici sorprendenti: è in grado di "guardare" l'interfaccia di un computer o di uno smartphone per capire come usarla, oppure può osservare lo schizzo grafico di un sito web e scrivere in automatico il codice informatico per costruirlo. [17] [18]

6.2.1 Training

Qwen3-VL è un modello puramente linguistico: non calcola coordinate geometriche, ma genera semplici stringhe di testo (token) osservando un'immagine.

Questo significa che ogni forma di addestramento applicata è, tecnicamente, l'addestramento di un modello linguistico. Non potendo utilizzare una classica funzione di errore spaziale, come la "loss di detection" tipica della computer vision, l'unica vera scelta riguarda l'origine del segnale usato per valutare e correggere le risposte del modello. Per questo motivo, sono state adottate due diverse politiche di apprendimento.

LoRA

Prima di analizzare le politiche di addestramento, occorre chiarire come renderlo computazionalmente sostenibile.

La tecnica LoRA (Low-Rank Adaptation) nasce da una semplice intuizione: per adattare un modello serve solo una piccola correzione mirata, non una riscrittura da zero. Invece di modificare milioni di parametri originali, i quali vengono "congelati". LoRA concentra l'aggiornamento in due tabelle leggerissime, con poche migliaia di valori che agiscono come un filtro correttivo. A fine addestramento, queste tabelle vengono fuse nel modello di base, mantenendone inalterata la velocità di esecuzione. È importante precisare che LoRA non è un obiettivo di apprendimento (non decide cosa il modello deve imparare), ma una pura tecnica di efficienza (decide quali parti aggiornare). Per questo può essere abbinata a qualsiasi metodo di addestramento, compresi quelli descritti nei paragrafi successivi.

Cross-Entropy

Nel fine-tuning supervisionato disponiamo, per ogni immagine, della risposta esatta che vorremmo far produrre al modello.

Conoscendo in anticipo il risultato atteso, possiamo fornire al modello l'intera sequenza corretta insieme all'immagine e alla domanda, sfruttando una tecnica nota come teacher forcing. Invece di fargli generare il testo parola per parola, il modello elabora tutta la sequenza simultaneamente in un unico passaggio, calcolando quale probabilità avrebbe assegnato a ciascuna parola corretta. Da questo calcolo si ricava il margine di errore, che viene poi utilizzato in un unico passaggio a ritroso per aggiornare le matrici di LoRA.

Tuttavia, il modo in cui questo errore viene calcolato presenta un limite strutturale profondo, fondamentale per comprendere i risultati dei nostri esperimenti. Il sistema valuta le parole prodotte dal modello come un vocabolario di simboli del tutto distinti, tra i quali non esiste un concetto di vicinanza. Nella formula di base, il modello non ha modo di intuire che quei numeri rappresentano una posizione nello spazio e che, geometricamente parlando, "sbagliare di poco" è infinitamente meglio che "sbagliare di molto".

GRPO

Tramite Reinforcement Learning si confrontano i contorni disegnati dal modello con quelli reali, assegnando un punteggio da 0 a 1 che penalizza il sistema se dimentica oggetti, ne inventa di inesistenti o li inquadra in modo impreciso.

Per capire se un punteggio sia effettivamente buono, abbiamo utilizzato la tecnica GRPO (Group Relative Policy Optimization). Il principio è semplice: il modello genera diverse risposte alternative per la stessa immagine. Quelle che ottengono un punteggio superiore alla media del gruppo vengono premiate e rinforzate, mentre quelle sotto la media vengono scoraggiate.

Il prezzo da pagare per questa libertà è puramente computazionale. Poiché il modello deve materialmente generare diverse risposte parola per parola, invece di leggerne una sola in simultanea.

Risultati

Le tre configurazioni confrontate sono cumulative: Base è il modello pre-addestrato senza alcun adattamento, C-E aggiunge il fine-tuning supervisionato con cross-entropy, GRPO applica sopra quest'ultimo un ulteriore stadio di apprendimento per rinforzo guidato da una ricompensa basata sull'IoU.

	P	R	<i>F1</i>	AP₅₀	AP_{50:95}
Base	0.9020	0.9460	0.9235	0.8556	0.5746
C-E	0.9387	0.9287	0.9337	0.8878	0.5586
GRPO	0.9360	0.9332	0.9346	0.8884	0.5610

I risultati dei test mostrano due tendenze opposte: l'adattamento migliora nettamente la precisione generale riducendo i falsi positivi, ma peggiora l'accuratezza geometrica dei contorni.

Questo accade perché il sistema di addestramento non comprende la vicinanza spaziale: per il modello, sbagliare una coordinata di un pixel o di cento comporta la stessa penalità. Impara quindi molto bene cosa identificare e quanti oggetti ci sono, ma non dove tracciarne esattamente i bordi.

Inoltre, abbiamo notato che il modello adattato diventa più prudente, rischiando di ignorare qualche oggetto reale pur di non generare falsi allarmi. L'apprendimento per rinforzo ha corretto in parte questo sbilanciamento, ma non ha migliorato il disegno dei contorni, a causa di parametri di addestramento risultati troppo conservativi.

In conclusione, l'adattamento di un VLM migliora enormemente la sua capacità di riconoscere il contenuto dell'immagine, ma non affina il ritaglio spaziale. La precisione millimetrica resta un suo limite strutturale, un compito in cui i rilevatori specializzati rimangono insuperabili.

6.3 Gemma 4

Sviluppato da Google, Gemma 4 è un modello che si distingue per la sua capacità di adattarsi a situazioni diverse. La sua innovazione principale è la capacità di guardare le immagini con livelli di "attenzione" regolabili. In pratica, se il compito richiede molta precisione, il modello può analizzare l'immagine in modo minuzioso; se invece serve rapidità, come quando deve elaborare i numerosi fotogrammi di un video, può dare un'occhiata più veloce e generale. [19]

Poiché Gemma 4 è un ottimo "pensatore" ma è meno adatto a calcolare con esattezza la posizione geometrica delle cose, spesso viene fatto lavorare in squadra con programmi visivi più classici. In questa collaborazione, il programma tradizionale fa il lavoro manuale (come isolare gli oggetti nella foto), mentre Gemma 4 fa da "mente", analizzando il risultato e scrivendo un resoconto comprensibile a parole.

6.3.1 Risultati

Per verificare se una maggiore capacità di elaborazione potesse superare i limiti evidenziati, abbiamo confrontato Qwen3-VL (8 miliardi di parametri) con la versione di Gemma 4 da 31 miliardi di parametri. La scelta di impiegare modelli di sviluppatori differenti ha un fine puramente esplorativo: l'obiettivo è analizzare le potenzialità e i limiti strutturali della tecnologia VLM nel suo complesso, senza alcuna intenzione di focalizzarsi su una competizione tra produttori.

Mantenendo l'addestramento tramite cross-entropy (come fatto per Qwen3-VL), i risultati di Gemma 4 si sono rivelati inaspettatamente simili a quelli del modello più piccolo. L'aumento esponenziale dei parametri non ha prodotto alcun salto qualitativo significativo, mostrando solo variazioni marginali: una leggera diminuzione della precision a fronte di un lieve miglioramento della recall.

Questo esito conferma che il collo di bottiglia nella localizzazione non risiede nella dimensione del modello, ma nel metodo di apprendimento. Poiché la cross-entropy tratta le coordinate come semplici stringhe di testo prive di nozioni spaziali, un modello più intelligente non possiede comunque gli strumenti matematici per disegnare contorni migliori. Va tuttavia precisato che l'esperimento è stato condotto su un dataset di dimensioni ridotte; è quindi probabile che la scarsità di esempi abbia penalizzato il modello da 31 miliardi, limitandone il pieno potenziale computazionale.

In sintesi, pur tenendo conto del limite imposto dai dati, l'esperimento dimostra che scalare i parametri da 8B a 31B si rivela inefficace se la funzione di loss non è strutturalmente adatta al compito geometrico richiesto. Aumentare le dimensioni di un VLM, da solo, non basta a trasformarlo in un rilevatore spaziale di precisione.

7. Grounding DINO

Storicamente, l'Object Detection è stata dominata da reti neurali convoluzionali come YOLO. Pur avendo rivoluzionato il settore grazie all'elaborazione ultra-rapida dell'immagine in un singolo passaggio per restituire i bounding box, YOLO è un sistema Closed-Set, cioè è capace di riconoscere solo le categorie per cui è stato esplicitamente addestrato e fallisce su oggetti al di fuori di questo vocabolario. [20]

Per superare questo limite, sono nati modelli multimodali Open-Set come Grounding DINO. Il termine "grounding" indica la capacità di ancorare un concetto testuale a un'entità fisica nell'immagine. Fornendo un'immagine e una descrizione a parole, il modello estrae le caratteristiche visive e testuali tramite due Backbone dedicati, entrambi basati sulla tecnologia Transformer. Utilizzando poi meccanismi di Cross-Attention, fa "dialogare" queste fonti: l'immagine si allinea al significato delle parole e viceversa. Il risultato è la generazione di bounding box precisi per qualsiasi oggetto descritto, senza necessità di riaddestramento. [21]

Mentre Grounding DINO eccelle nel rilevamento tramite bounding box, un'altra evoluzione parallela basata sui Transformer è il Segment Anything Model (SAM). Pur condividendo tecnologie simili, i due si differenziano per obiettivi e risoluzione spaziale:

- L'obiettivo geometrico (YOLO e Grounding DINO): approccio sintetico incentrato sulle coordinate esterne, ideale per il conteggio o il tracciamento rapido nella scena.
- L'obiettivo topologico (SAM): non ci si limita a inscatolare l'oggetto, ma genera una maschera esatta che ne segue i contorni reali.

In sintesi, la Computer Vision odierna offre strumenti complementari: l'efficienza di YOLO per contesti chiusi e noti; la flessibilità open-set di Grounding DINO per localizzare concetti testuali; e la precisione al pixel di SAM per estrarre le forme nel dettaglio.

Bibliografia

- [1] Wikipedia. *Rete Neurale*. URL: https://it.wikipedia.org/wiki/Rete_neurale_artificiale.
- [2] Wikipedia Contributors. *Neural_Networks.svg*. URL: [https://en.wikipedia.org/wiki/Neural_network_\(machine_learning\)#/media/File:Colored_neural_network.svg](https://en.wikipedia.org/wiki/Neural_network_(machine_learning)#/media/File:Colored_neural_network.svg).
- [3] IBM. *Modello di Reti neurali - IBM Documentation*. 2024. URL: <https://www.ibm.com/docs/it/spss-modeler/18.6.0?topic=networks-neural-model>.
- [4] IBM. *Model Training*. URL: <https://www.ibm.com/it-it/think/topics/model-training>.
- [5] Ultralytics. *Epoch in Machine Learning (ML)*. URL: <https://www.ultralytics.com/glossary/epoch>.
- [6] IBM. *Pytorch*. URL: <https://www.ibm.com/it-it/think/topics/pytorch>.
- [7] Ani Aggarwal. *YOLO*. URL: <https://medium.com/analytics-vidhya/yolo-explained-5b6f4564f31>.
- [8] Ultralytics. *Ultralytics YOLO*. URL: <https://docs.ultralytics.com/it/models/yolo26#ultralytics-yolo26>.
- [9] Wikipedia Contributors. *Precision&Recall.svg*. URL: https://en.wikipedia.org/wiki/Precision_and_recall.
- [10] Computer Vision Wiki. *Intersection over Union*. URL: <https://wiki.cloudfactory.com/docs/mp-wiki/metrics/iou-intersection-over-union>.
- [11] Computer Vision Wiki. *mean Average Precision*. URL: <https://wiki.cloudfactory.com/docs/mp-wiki/metrics/map-mean-average-precision>.
- [12] Ultralytics. *SAM3*. URL: <https://docs.ultralytics.com/models/sam-3>.
- [13] Alexey Dosovitskiy et al. *An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale*. URL: <https://arxiv.org/abs/2010.11929>.
- [14] Enze Xie et al. *SegFormer: Simple and Efficient Design for Semantic Segmentation with Transformers*. URL: <https://arxiv.org/abs/2105.15203>.

- [15] Meta AI. *Segment Anything Model 3 (SAM 3): Segment Anything with Concepts*. URL: <https://ai.meta.com/research/>.
- [16] Alibaba Cloud. *QwenLM/Qwen3-VL - Official GitHub Repository*. URL: <https://github.com/QwenLM/Qwen3-VL>.
- [17] Ollama. *qwen3-vl Model Library*. URL: <https://ollama.com/library/qwen3-vl>.
- [18] LM Studio. *Qwen3-VL Features and Visual Agent Capabilities*. URL: <https://lmstudio.ai>.
- [19] Google AI for Developers. *Vision understanding / Gemma*. URL: <https://ai.google.dev/gemma>.
- [20] Joseph Redmon et al. *You only look once: Unified, real-time object detection*. URL: <https://arxiv.org/abs/1506.02640>.
- [21] Shilong Liu et al. *Grounding DINO: Marrying DINO with Grounded Pre-Training for Open-Set Object Detection*. URL: <https://arxiv.org/abs/2303.05499>.